INFORME DE LABORATORIO

Actividad N° 01 · Semana 08

Curso: Base de Datos II

ESPEJO COSME JORCH

ORGANIZACIÓN DE ESQUEMAS LÓGICOS Y ESTRATEGIAS DE INDEXACIÓN

1 Esquemas y roles **2** Índice clúster **3** No clúster + INCLUDE **4** Índice compuesto **5** Fragmentación

1. CONSOLIDACIÓN Y VERIFICACIÓN DE ESQUEMAS LÓGICOS

¿Por qué separar en esquemas?

Un esquema es un contenedor lógico de objetos. Separar las entidades evita dejar todo en dbo y permite dar o negar permisos a nivel de SCHEMA, en lugar de tabla por tabla. Cada rol recibe solo lo estrictamente necesario: principio de mínimo privilegio (RBAC).

6 esquemas creados:

Academico Catalogo Documento Evaluacion Seguridad

Tramite

Roles y permisos aplicados

```
CREATE ROLE RolMesaDePartes;
-- Mesa de Partes
GRANT SELECT ON SCHEMA::Catalogo
        TO RolMesaDePartes;
GRANT SELECT, INSERT, UPDATE
        ON SCHEMA::Tramite TO RolMesaDePartes;
-- Bloqueo al esquema sensible
DENY SELECT, INSERT, UPDATE, DELETE
      ON SCHEMA::Seguridad TO RolMesaDePartes;
```

Matriz de permisos resultante (consulta sobre sys.database_permissions)

Rol corporativo	Catalogo	Academico	Tramite	Evaluacion	Seguridad
RolMesaDePartes	SELECT	—	S · I · U	—	DENY
RolComiteEvaluador	—	SELECT	—	S · I · U	DENY
RolAuditorSeguridad	—	—	—	—	SELECT

SQL

```
USE GradosTitulos;
GO
-- Creación y validación de esquemas lógicos corporativos
IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Catalogo')
    EXEC('CREATE SCHEMA Catalogo;');

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Academico')
    EXEC('CREATE SCHEMA Academico;');

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Tramite')
    EXEC('CREATE SCHEMA Tramite;');

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Evaluacion')
    EXEC('CREATE SCHEMA Evaluacion;');

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Documento')
    EXEC('CREATE SCHEMA Documento;');

-- Creación de roles de base de datos
IF NOT EXISTS (SELECT * FROM sys.database_principals WHERE name = 'RolMesaDePartes')
    CREATE ROLE RolMesaDePartes;

IF NOT EXISTS (SELECT * FROM sys.database_principals WHERE name = 'RolComiteEvaluador')
    CREATE ROLE RolComiteEvaluador;

IF NOT EXISTS (SELECT * FROM sys.database_principals WHERE name = 'RolAuditorSeguridad')
    CREATE ROLE RolAuditorSeguridad;

-- Conceder y denegar permisos sobre esquemas
GRANT SELECT ON SCHEMA::Catalogo TO RolMesaDePartes;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Evaluacion TO RolMesaDePartes;

GRANT SELECT ON SCHEMA::Academico TO RolComiteEvaluador;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Evaluacion TO RolComiteEvaluador;

DENY SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Seguridad TO RolAuditorSeguridad;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Seguridad TO RolAuditorSeguridad;

-- Consultar los permisos de los roles
SELECT
    dp.name  AS RolResponsable,
    perm.state_desc  AS Accion,
    perm.permission_name AS Permiso,
    s.name AS EsquemaRequerido
FROM sys.database_permissions perm
INNER JOIN sys.database_principals dp ON perm.grantee_principal_id = dp.principal_id
INNER JOIN sys.schemas s ON perm.major_id = s.schema_id
WHERE perm.class_desc = 'SCHEMA'
ORDER BY dp.name, s.name;
```

No se encontraron problemas.

RESULTADO

Resultados | Mensajes

	RolCorporativo	Accion	Permiso	EsquemaProtegido
1	RolAuditorSeguridad	GRANT	SELECT	Seguridad
2	RolComiteEvaluador	GRANT	SELECT	Academico
3	RolComiteEvaluador	GRANT	INSERT	Evaluacion
4	RolComiteEvaluador	GRANT	SELECT	Evaluacion
5	RolComiteEvaluador	GRANT	UPDATE	Evaluacion
6	RolComiteEvaluador	DENY	DELETE	Seguridad
7	RolComiteEvaluador	DENY	INSERT	Seguridad
8	RolComiteEvaluador	DENY	SELECT	Seguridad
9	RolComiteEvaluador	DENY	UPDATE	Seguridad
10	RolMesaDePartes	GRANT	SELECT	Catalogo
11	RolMesaDePartes	DENY	DELETE	Seguridad
12	RolMesaDePartes	DENY	INSERT	Seguridad
13	RolMesaDePartes	DENY	SELECT	Seguridad
14	RolMesaDePartes	DENY	UPDATE	Seguridad
15	RolMesaDePartes	GRANT	INSERT	Tramite
16	RolMesaDePartes	GRANT	SELECT	Tramite
17	RolMesaDePartes	GRANT	UPDATE	Tramite

2.ÍNDICES CLÚSTER ESTRATÉGICOS: SOLO PUEDE EXISTIR UNO

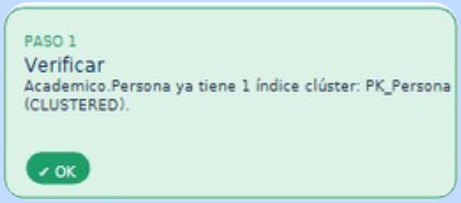
Árbol B de un índice clúster

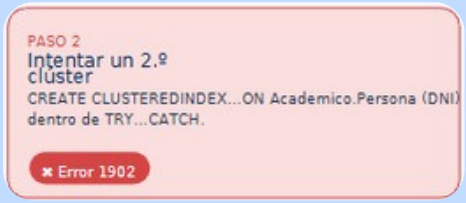


1 **El nivel hoja contiene los datos reales**
No guarda punteros:el índice clúster es la tabla ordenada por la clave elegida.

2 **Imposible un doble orden físico**
Las filas solo pueden almacenarse en una secuencia dentro de las páginas de datos.

3 **Para otros órdenes: índices No Clúster**
Estructuras separadas que apuntan a la clave del clúster (por ejem. DNI o Fecha Inicio).







SQL

```sql
USE GradosTitulos;
GO
-- ============================================================
-- PASO 1: Verificar que la tabla Academico.Persona ya tiene 1 índice Clúster
-- (Creado en su clave primaria IdPersona)
-- ============================================================
SELECT
    s.name AS Esquema,
    t.name AS Tabla,
    i.name AS NombreIndice,
    i.type_desc AS TipoIndice
FROM sys.indexes i
INNER JOIN sys.tables t ON i.object_id = t.object_id
INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE s.name = 'Academico'
    AND t.name = 'Persona'
    AND i.type = 1; -- 1 = CLUSTERED
GO
-- ============================================================
-- PASO 2: Demostración del límite físico (Intentar crear un 2do índice Clúster)
-- SQL Server bloqueará la operación porque las tablas ya están ordenadas por 1
-- ============================================================
BEGIN TRY
    CREATE CLUSTERED INDEX IX_Persona_SegundoClustered
    ON Academico.Persona (DNI);
END TRY
BEGIN CATCH
    SELECT
        ERROR_NUMBER() AS NumeroError,
        ERROR_MESSAGE() AS MensajeDelMotorSQL;
END CATCH;
GO

-- ============================================================
-- PASO 3: Solución correcta cuando ya existe un índice Clúster
-- Para ordenar por una segunda columna (DNI), se usa un índice NONCLUSTERED
-- ============================================================
IF NOT EXISTS (
    SELECT 1 FROM sys.indexes
    WHERE name = 'IX_Persona_DNI_Demo'
        AND object_id = OBJECT_ID('Academico.Persona')
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Persona_DNI_Demo
        ON Academico.Persona (DNI);
END
GO
```

RESULTADO

Resultados | Mensajes

	Esquema	Tabla	NombreIndice	TipoIndice
1	Academico	Persona	PK_Persona	CLUSTERED

	NumeroError	MensajeDelMotorSQL
1	1902	Cannot create more than one clustered

3.ÍNDICES NO CLÚSTER CON COLUMNAS INCLUIDAS (INCLUDE)

Dos índices cubiertos creados sobre Academico.Persona

```
CREATE NONCLUSTERED INDEX IX_Persona_DNI_Cover
ON Academico.Persona (DNI)
INCLUDE (Nombres, ApellidoPaterno,
         ApellidoMaterno, Correo, TipoPersona);
```

```
CREATE NONCLUSTERED INDEX IX_Persona_TipoPersona
ON Academico.Persona (TipoPersona)
INCLUDE (DNI, Nombres, ApellidoPaterno,
         ApellidoMaterno);
```

Índice cubierto con INCLUDE

1 Index Seek

Obtiene todas las columnas pedidas directamente desde el nivel hoja del índice, sin tocar la tabla base. Menos lecturas lógicas (I/O) y menos CPU.

Rendimiento: MUY SUPERIOR

Consulta con Key Lookup

Index Seek + Key Lookup x fila

Tras buscar en el índice no clúster, hace lecturas aleatorias adicionales a la tabla base (clúster) por cada fila recuperada para traer las columnas faltantes. Multiplica el costo de I/O y CPU.

Rendimiento: PENALIZADO

¿Por qué solo aparece «Los comandos se han completado correctamente»?

Porque CREATE NONCLUSTERED INDEX es DDL (crea estructura), no una consulta de lectura. El SELECT * FROM sys.indexes dentro de IF NOT EXISTS solo verifica si el índice ya existe para evitar errores de duplicado al re-ejecutar el script.

SQL

SQLQuery5.sql - localhost.GradosTitulos (sa (67))*

```sql
USE GradosTitulos;
GO
-- Índice No Clúster con cobertura para búsquedas por DNI
IF NOT EXISTS (
      SELECT * FROM sys.indexes
      WHERE name = 'IX_Persona_DNI_Cover'
            AND object_id = OBJECT_ID('Academico.Persona')
)
BEGIN
      CREATE NONCLUSTERED INDEX IX_Persona_DNI_Cover
      ON Academico.Persona (DNI)
      INCLUDE (Nombres, ApellidoPaterno, ApellidoMaterno, Correo, TipoPersona);
END
GO

-- Índice No Clúster para filtrado por Tipo de Persona
IF NOT EXISTS (
      SELECT * FROM sys.indexes
      WHERE name = 'IX_Persona_TipoPersona'
            AND object_id = OBJECT_ID('Academico.Persona')
)
BEGIN
      CREATE NONCLUSTERED INDEX IX_Persona_TipoPersona
      ON Academico.Persona (TipoPersona)
      INCLUDE (DNI, Nombres, ApellidoPaterno, ApellidoMaterno);
END
GO

-- Índice No Clúster para búsqueda por Tipo de Persona (ya existe un índice NoClustered)
IF NOT EXISTS (
      SELECT * FROM sys.indexes
      WHERE name = 'IX_Persona_TipoPersona'
            AND object_id = OBJECT_ID('Academico.Persona')
)
BEGIN
      CREATE NONCLUSTERED INDEX IX_Persona_TipoPersona
      ON Academico.Persona (TipoPersona)
      INCLUDE (DNI, Nombres, ApellidoPaterno, ApellidoMaterno);
END
GO
```

RESULTADO

Mensajes

```
Los comandos se han completado correctamente.

Hora de finalización: 2026-10-22T13:51:12.5150695-05:00
```

4.INDEXACIÓN COMPUESTA PARA CONSULTAS ANALÍTICAS DE TRÁMITES

IX_Tramite_Estado_Fecha

ON Tramite.Tramite ·reportesdetrámites por Estado y rango de fechas

COLUMNAS CLAVE (orden)

1 IdEstadoTramite 2 FechaInicio

INCLUDE (cobertura)

IdMatricula IdTipoTramite IdCoordinacion

IX_Pago_Tramite_Fecha

ON Tramite.Pago · pagosportrámite y fecha

COLUMNAS CLAVE (orden)

1 IdTramite 2 FechaPago

INCLUDE (cobertura)

Monto NroRecibo TipoPago

Cómo pensar un índice compuesto

Analiza el patrón de consulta: qué columnas van en igualdad (Estado), cuáles en rangos (FechaInicio) y cuáles solo se muestran. El orden de las columnas clave importa: (A, B) sirve distinto que (B, A). No basta con ordenar por selectividad.

Ejemplo de la clase: (ClienteID, FechaPedido) favorece «pedidos de un cliente en un rango de fechas».

SQL

SQLQuery6.sql - localhost.GradosTitulos (sa (67))*

```
USE GradosTitulos;
GO

-- Índice compuesto para reportes analíticos de trámites por Estado y Rango de Fecha
IF NOT EXISTS (
    SELECT * FROM sys.indexes
    WHERE name = 'IX_Tramite_Estado_Fecha'
      AND object_id = OBJECT_ID('Tramite.Tramite')
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Tramite_Estado_Fecha
      ON Tramite.Tramite (IdEstadoTramite, FechaInicio)
      INCLUDE (IdMatricula, IdTipoTramite, IdCoordinacion);
END
GO

-- Índice para agilizar la consulta de pagos por trámite y fecha
IF NOT EXISTS (
    SELECT * FROM sys.indexes
    WHERE name = 'IX_Pago_Tramite_Fecha'
      AND object_id = OBJECT_ID('Tramite.Pago')
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Pago_Tramite_Fecha
      ON Tramite.Pago (IdTramite, FechaPago)
      INCLUDE (Monto, NroRecibo, TipoPago);
END
GO
```

RESULTADO

Mensajes

```
Los comandos se han completado correctamente.

Hora de finalización: 2026-10-22T13:57:58.3524036-05:00
```

5.AUDITORÍA, MONITOREO Y ANÁLISIS DE FRAGMENTACIÓN

1 · Auditoría de índices

Lista cadaíndiceporesquemaytabla con sys.indexes + sys.tables + sys.schemas.

```
SELECT s.name, t.name, i.name,
       i.type_desc, i.is_primary_key,
       i.is_unique
FROM sys.indexes i ... WHERE i.name IS NOT NULL;
```

2 · Fragmentación física

Mide el%defragmentaciónyeltamaño de cada índice con la vista de administración dinámica.

```
FROM sys.dm_db_index_physical_stats(
     DB_ID(), NULL, NULL, NULL, 'LIMITED')
WHERE avg_fragmentation_in_percent > 10
ORDER BY avg_fragmentation_in_percent DESC;
```

¿Qué hacer según el % de fragmentación?

< 10 %	10 % – 30 %	> 30 %
Sin acción Elíndiceestáen buen estado.	REORGANIZE Operaciónliviana: desfragmenta solo el nivel hoja reutilizando páginas; poco CPU y sin bloqueos prolongados.	REBUILD Creaelíndice nuevo en disco, compacta páginas y actualiza estadísticas; devuelve el rendimiento óptimo.

SQL

SQLQuery1.sql - local...GradosTitulos (sa (67))*

```sql
USE GradosTitulos;
GO

-- 1. Auditoria general de todos los índices creados por esquema y tabla
SELECT
     s.name AS Esquema,
     t.name AS Tabla,
     i.name AS NombreIndice,
     i.type_desc AS TipoIndice,
     i.is_primary_key AS EsClavePrimaria,
     i.is_unique AS EsUnico
FROM sys.indexes i
     INNER JOIN sys.tables t ON i.object_id = t.object_id
     INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
     WHERE i.name IS NOT NULL
     ORDER BY s.name, t.name, i.type;
GO

-- 2. Análisis de fragmentación física de los índices en el sistema
SELECT
     OBJECT_SCHEMA_NAME(ps.object_id) AS Esquema,
     OBJECT_NAME(ps.object_id) AS Tabla,
     i.name AS Indice,
     ps.avg_fragmentation_in_percent AS PorcentajeFragmentacion,
     ps.page_count AS CantidadPaginas
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ps
     INNER JOIN sys.indexes i ON ps.object_id = i.object_id AND ps.index_id = i.index_id
WHERE ps.avg_fragmentation_in_percent > 10
ORDER BY ps.avg_fragmentation_in_percent DESC;
GO
```

RESULTADO

Resultados | Mensajes

	Esquema	Tabla	NombreIndice	TipoIndice	EsClavePrimaria	EsUnico
1	Academico	Docente	PK_Docente	CLUSTERED	1	1
2	Academico	Docente	UQ_Docente_Persona	NONCLUSTERED	0	1
3	Academico	Facultad	PK_Facultad	CLUSTERED	1	1
4	Academico	Facultad	UQ_Facultad_Nombre	NONCLUSTERED	0	1
5	Academico	GradoTituloCatalogo	PK_GradoTituloCatalogo	CLUSTERED	1	1
6	Academico	Matricula	PK_Matricula	CLUSTERED	1	1
7	Academico	Persona	PK_Persona	CLUSTERED	1	1
8	Academico	Persona	UQ_Persona_DNI	NONCLUSTERED	0	1
9	Academico	Persona	IX_Persona_DNI_Demo	NONCLUSTERED	0	0
10	Academico	Persona	IX_Persona_DNI_Cover	NONCLUSTERED	0	0
11	Academico	Persona	IX_Persona_TipoPersona	NONCLUSTERED	0	0
12	Academico	ProgramaEstudios	PK_ProgramaEstudios	CLUSTERED	1	1
13	Academico	ProgramaEstudios	UQ_Programa_Nombre_Modalidad	NONCLUSTERED	0	1
14	Catalogo	EstadoTramite	PK_EstadoTramite	CLUSTERED	1	1
15	Catalogo	EstadoTramite	UQ_EstadoTramite_Descripcion	NONCLUSTERED	0	1
16	Catalogo	LineaInvestigacion	PK_LineaInvestigacion	CLUSTERED	1	1